# PreLAB: Timer Interrupt & PWM

## Introduction

In this prelab, you will learn how to set up MCU Timers for Timer Interrupt and PWM output.

## Part 1: Timer Interrupt

### A. Register List

List of TIMx registers for Timer Interrupt

| Type | Register Name | Description |
|---|---|---|
| TIMx | TIMx_ CR1 | TIMx control register 1 |
| | TIMx_ PSC | TIMx prescaler register |
| | TIMx_ARR | TIMx auto-reload register |
| | TIMx_DIER | TIMx DMA Interrupt Enable register |

### B. Register Setting

1. **System Clock setting**

- RCC setting (PLL)

2. **Timer Counter setting**

- Enable Timer Peripheral Clock (RCC->APB1ENR)
- Set Timer Clock Pre-scaler value (TIMx->PSC : PSC[15:0])
- Set Auto-reload value (TIMx->ARR : ARR)
- Set Counting Direction (TIMx->CR1 : DIR)
- Enable counter (TIMx->CR1 : CEN)

3. **Timer Interrupt setting**

- Enable Timer DMA/Interrupt. (TIMx->DIER : UIE)

4. **NVIC setting**

- Set Interrupt Priority NVIC_SetPriority(TIMx_IRQn,2)
- Enable TIMx Interrupt: NVIC_EnableIRQ(TIMx_IRQn)

### C. Exercise

**Exercise: TIMER Library**

- Download the header library files and save under `include\`.

    - `ecTIM2_student.h, ecTIM2_student.c`: Click here to download

- Rename the files as `ecTIM2.h, ecTIM2.c`

  > Write your name and modified date in the comment box

- Fill in the empty spaces in the downloaded library: `ecTIM2.c`
- Run the program and check your result.
- **Submit** the exercise code in the report

**Exercise Code**

```
// Search for [YOUR CODE GOES HERE !!!!]  and fill in the code.
// Exercises are in these functions.
void TIM_init(TIM_TypeDef* TIMx, uint32_t msec);
void TIM_init_us(TIM_TypeDef* TIMx, uint32_t usec);
void TIM_period_us(TIM_TypeDef* TIMx, uint32_t usec);
```

Exercise

```
////////////////////////////////////////////////////////////////
/*                                          EXERCISE
*/
////////////////////////////////////////////////////////////////
/*
// Search for [YOUR CODE GOES HERE !!!!]  and fill in the code.
// Exercises are in these functions.
void TIM_init(TIM_TypeDef* TIMx, uint32_t msec);
void TIM_init_us(TIM_TypeDef* TIMx, uint32_t usec);
void TIM_period_us(TIM_TypeDef* TIMx, uint32_t usec);
*/

// Timer Counter Initialization
// Default : f_cnt =  10kHz, msec= 1 to 6,553 msec
void TIM_init(TIM_TypeDef* TIMx, uint32_t msec){
// 1. Enable Timer CLOCK
    TIM_clock_enable(TIMx);

// 2. Set CNT period
    TIM_period_ms(TIMx, msec);

// 3. COUNTER Direction
    // Up-Counter
    // TIMx->CR1 ________________;
    // [YOUR CODE GOES HERE !!!!]

    // Down-Counter
    //TIMx->CR1 |= TIM_CR1_DIR;
```

```
// 4. Enable Timer Counter
    TIMx->CR1 |= TIM_CR1_CEN;
}


// Timer Counter Initialization
// Default : f_cnt =  1 MHz, usec= 1 to 65,530 usec
void TIM_init_us(TIM_TypeDef* TIMx, uint32_t usec){
// 1. Enable Timer CLOCK
    // [YOUR CODE GOES HERE !!!!]

// 2. Set CNT period
    TIM_period_us(TIMx, usec);

// 3. CNT Direction
    // [YOUR CODE GOES HERE !!!!]

// 4. Enable Timer Counter
    // [YOUR CODE GOES HERE !!!!]
}


// Limits    : 16-bit ARR -> f_cnt =  1 MHz, usec= 1 to 65,530 usec (ARR < 0xFFFF)
void TIM_period_us(TIM_TypeDef* TIMx, uint32_t usec){

    uint32_t Sys_CLK = EC_SYSCLK;                   // 84MHz (PLL) or 16MHz (HSI)
    uint32_t ARRval=0;

    // Counter Frequency (f_cnt)
    uint32_t f_cnt   =  1000000UL;                  // 16-bit CNT: f_cnt=1MHz
(default)

    // Prescaler Value
    uint32_t PSCval = Sys_CLK / f_cnt;              // 84MHz -> 1 MHz (PSC=84)
    // TIMx->PSC = _________________;
    // [YOUR CODE GOES HERE !!!!]

    // Reload Value (ARR) for msec unit
    // uint32_t ARRval = ____________________;
    // [YOUR CODE GOES HERE !!!!]
    TIMx->ARR = ARRval - 1;
}
```

ecTIM2.h

```
////////////////////////////////////////////
/* Timer Update Interrupt Configuration     */
////////////////////////////////////////////

// Timer Update Interrupt Initialization
void TIM_UI_init(TIM_TypeDef* TIMx, uint32_t msec);     // 16-bit  msec < 65,530
msec
```

```
void TIM_UI_init_us(TIM_TypeDef* TIMx, uint32_t usec);  // 16-bit  usec < 65,530
usec

// Timer Update Interrupt On/OFf
void TIM_UI_enable(TIM_TypeDef* TIMx);
void TIM_UI_disable(TIM_TypeDef* TIMx);

// Timer Update Interrupt Flag
uint32_t is_UIF(TIM_TypeDef *TIMx);
void clear_UIF(TIM_TypeDef *TIMx);


////////////////////////////////////////////////////////////
/* Timer Counter Initialization                           */
////////////////////////////////////////////////////////////

// Timer Counter Initialization
void TIM_init(TIM_TypeDef *TIMx, uint32_t msec);
void TIM_init_us(TIM_TypeDef *TIMx, uint32_t usec);

// Timer Counter Period Config
void TIM_period(TIM_TypeDef* TIMx, uint32_t msec);
void TIM_period_ms(TIM_TypeDef* TIMx, uint32_t msec);   // same as TIM_period()
void TIM_period_us(TIM_TypeDef* TIMx, uint32_t usec);

// Timer Counter Auxiliary Functions
uint32_t get_TIM_IRQn(TIM_TypeDef* TIMx);
void TIM_clock_enable(TIM_TypeDef* TIMx);
```

filled in Exercise ecTIM2.c code

```
/*----------------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : [Choi Junghyun]
Created          : 05-03-2021
Modified         : 10-29-2026 [WRITE THE DATE!!!!]
Language/ver     : C++ in VS Code

Description      : [Write description here!!]
/----------------------------------------------------------------*/

#include "ecTIM2.h"

////////////////////////////////////////////////////////////
/*                      EXERCISE                          */
////////////////////////////////////////////////////////////
/*
// Search for [YOUR CODE GOES HERE !!!!]  and fill in the code.
// Exercises are in these functions.
void TIM_init(TIM_TypeDef* TIMx, uint32_t msec);
```

```
void TIM_init_us(TIM_TypeDef* TIMx, uint32_t usec);
void TIM_period_us(TIM_TypeDef* TIMx, uint32_t usec);
*/


////////////////////////////////////////////
/* Timer Update Interrupt Configuration     */
////////////////////////////////////////////


void TIM_UI_init(TIM_TypeDef* TIMx, uint32_t msec){
// 1. Initialize Timer
    TIM_init(TIMx,msec);

// 2. Enable Update Interrupt
    TIM_UI_enable(TIMx);

// 3. NVIC Setting
    uint32_t IRQn_reg = get_TIM_IRQn(TIMx);
    NVIC_EnableIRQ(IRQn_reg);
    NVIC_SetPriority(IRQn_reg,2);
}

void TIM_UI_init_us(TIM_TypeDef* TIMx, uint32_t usec){
// 1. Initialize Timer
    TIM_init_us(TIMx,usec);

// 2. Enable Update Interrupt
    TIM_UI_enable(TIMx);

// 3. NVIC Setting
    uint32_t IRQn_reg = get_TIM_IRQn(TIMx);
    NVIC_EnableIRQ(IRQn_reg);
    NVIC_SetPriority(IRQn_reg,2);
}


// Enable Timer Update Interrupt
void TIM_UI_enable(TIM_TypeDef* TIMx){
    TIMx->SR = ~TIM_SR_UIF;
    TIMx->DIER |= TIM_DIER_UIE;
}

// Disable Timer Update Interrupt
void TIM_UI_disable(TIM_TypeDef* TIMx){
    TIMx->DIER &= ~(TIM_DIER_UIE);
}

uint32_t is_UIF(TIM_TypeDef *TIMx){
    return TIMx->SR & TIM_SR_UIF;
}
void clear_UIF(TIM_TypeDef *TIMx){
    TIMx->SR &= ~TIM_SR_UIF;
}
```

```
////////////////////////////////////////////////////////////////
/* Timer Counter Initialization                               */
////////////////////////////////////////////////////////////////


// Timer Counter Initialization
// Default : f_cnt =  10kHz, msec= 1 to 6,553 msec
void TIM_init(TIM_TypeDef* TIMx, uint32_t msec){
// 1. Enable Timer CLOCK
    TIM_clock_enable(TIMx);

// 2. Set CNT period
    TIM_period_ms(TIMx, msec);

// 3. COUNTER Direction
    // Up-Counter
    // TIMx->CR1 ___________________;
    TIMx->CR1 &= ~TIM_CR1_DIR;

    // Down-Counter
    //TIMx->CR1 |= TIM_CR1_DIR;

// 4. Enable Timer Counter
    TIMx->CR1 |= TIM_CR1_CEN;
}


// Timer Counter Initialization
// Default : f_cnt =  1 MHz, usec= 1 to 65,530 usec
void TIM_init_us(TIM_TypeDef* TIMx, uint32_t usec){
// 1. Enable Timer CLOCK
    TIM_clock_enable(TIMx);

// 2. Set CNT period
    TIM_period_us(TIMx, usec);

// 3. CNT Direction
    TIMx->CR1 &= ~TIM_CR1_DIR;

// 4. Enable Timer Counter
    TIMx->CR1 |= TIM_CR1_CEN;
}


// Limits    : 16-bit ARR -> f_cnt =  10kHz, msec= 1 to 6,553 msec (ARR < 0xFFFF)
void TIM_period(TIM_TypeDef* TIMx, uint32_t msec){

    uint32_t Sys_CLK = EC_SYSCLK;                   // 84MHz (PLL) or 16MHz (HSI)

    // Counter Frequency (f_cnt)
```

```
    uint32_t f_cnt   =  10000UL;                    // 16-bit CNT: f_cnt=10kHz
(default)

    // Prescaler Value
    uint32_t PSCval = Sys_CLK / f_cnt;              // 84MHz -> 10 kHz (PSC=8400)
    TIMx->PSC = PSCval - 1;

    // Reload Value (ARR) for msec unit
    uint32_t ARRval = (f_cnt / 1000UL) * msec;      // Reload values needed for
msec
    TIMx->ARR = ARRval - 1;
}


// Limits    : 16-bit ARR -> f_cnt =  1 MHz, usec= 1 to 65,530 usec (ARR < 0xFFFF)
void TIM_period_us(TIM_TypeDef* TIMx, uint32_t usec){

    uint32_t Sys_CLK = EC_SYSCLK;                   // 84MHz (PLL) or 16MHz (HSI)
    uint32_t ARRval=0;

    // Counter Frequency (f_cnt)
    uint32_t f_cnt   =  1000000UL;                  // 16-bit CNT: f_cnt=1MHz
(default)

    // Prescaler Value
    uint32_t PSCval = Sys_CLK / f_cnt;              // 84MHz -> 1 MHz (PSC=84)
    // TIMx->PSC = _________________;
    TIMx->PSC = PSCval - 1;

    // Reload Value (ARR) for msec unit
    // uint32_t ARRval = ____________________;
    uint32_t ARRval = (f_cnt / 1000UL) * usec;
    TIMx->ARR = ARRval - 1;
}




void TIM_clock_enable(TIM_TypeDef* TIMx){
    if(TIMx ==TIM1) RCC->APB2ENR |= RCC_APB2ENR_TIM1EN;
    else if(TIMx ==TIM2) RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;
    else if(TIMx ==TIM3) RCC->APB1ENR |= RCC_APB1ENR_TIM3EN;
    else if(TIMx ==TIM4) RCC->APB1ENR |= RCC_APB1ENR_TIM4EN;
    else if(TIMx ==TIM5) RCC->APB1ENR |= RCC_APB1ENR_TIM5EN;
    else if(TIMx ==TIM9) RCC->APB2ENR |= RCC_APB2ENR_TIM9EN;
    else if(TIMx ==TIM10) RCC->APB2ENR |= RCC_APB2ENR_TIM10EN;
    else if(TIMx ==TIM11) RCC->APB2ENR |= RCC_APB2ENR_TIM11EN;
}



uint32_t get_TIM_IRQn(TIM_TypeDef* TIMx){
    uint32_t IRQn_reg = 0x00000000;
    if(TIMx == TIM1)       IRQn_reg = TIM1_UP_TIM10_IRQn;
```

```
    else if(TIMx == TIM2)  IRQn_reg = TIM2_IRQn;
    else if(TIMx == TIM3)  IRQn_reg = TIM3_IRQn;
    else if(TIMx == TIM4)  IRQn_reg = TIM4_IRQn;
    else if(TIMx == TIM5)  IRQn_reg = TIM5_IRQn;
    else if(TIMx == TIM9)  IRQn_reg = TIM1_BRK_TIM9_IRQn;
    else if(TIMx == TIM10) IRQn_reg = TIM1_UP_TIM10_IRQn;
    else if(TIMx == TIM11) IRQn_reg = TIM1_TRG_COM_TIM11_IRQn;
    return IRQn_reg;
}
```

### Exercise: TIMER application

This is an example code for Timer Interrupt that turns LED on/off at 1 second period.

### Configuration

- TIM2: Up-counting, Timer2_CLK = 100 kHz, COUNT_CLK = 1 kHz
- Use timer interrupt “void TIM2_IRQHandler(void)”

### Procedure

- Create a new project under the directory `EC\tutorial\`
  - Environment : `env:TU_TIM_Interrupt`
  - Source file: `TU_TIM_Interrupt_student.c`
- Modify the `platformio.ini` , to add new environment.
- Run the program and check your result.
- No need to submit the result in the prelab report
- But you need to show the demonstration in class.

TU_TIM_Interrupt_student.c

```
/*----------------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : [ YOUR NAME GOES HERE !!!!!]
Created          : 05-03-2021
Modified         : 09-03-2026 [WRITE THE DATE!!!!]
Language/ver     : C++ in VS Code

Description      : [Write description here!!]
/----------------------------------------------------------------*/


// #include "ecSTM32F411.h"
#include "stm32f411xe.h"
#include "ecRCC2.h"
#include "ecGPIO2.h"
#include "ecSysTick2.h"
#include "ecTIM2.h"
```

```c
#define LD2_PIN PA_5
uint32_t _count = 0;
// led_toggle() function should have been defined in ecGPIO2.c


// Initialization
void setup(void){
    RCC_PLL_init();
    SysTick_init();
    GPIO_init(LD2_PIN, OUTPUT);

    // TIM2 Update-Event Interrupt every 1 msec
    TIM_UI_init(TIM2, 1);           // UEV every 1 msec
    //TIM_UI_init_us(TIM2, 1000);   // UEV every 1000usec=1msec
}

int main(void) {
    // Initialization --------------------------------------------------
    setup();

    // Infinite Loop ---------------------------------------------------
    while(1){}
}

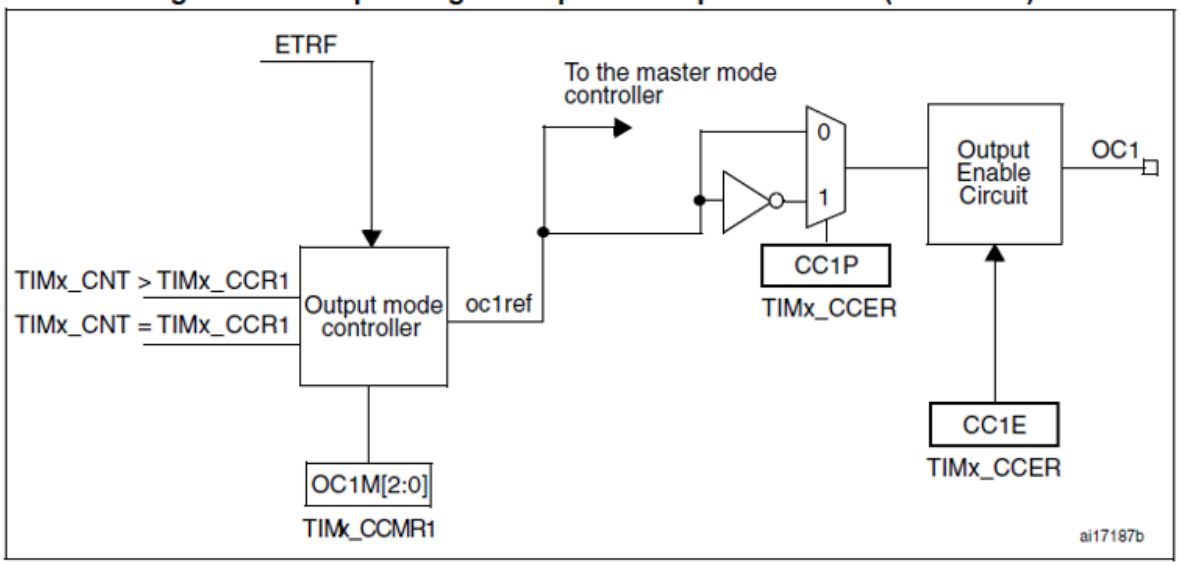

// TIM2 Update Interrupt Handler
void TIM2_IRQHandler(void){
    if(is_UIF(TIM2)){           // Check UIF(update interrupt flag)
        _count++;
        if (_count > 1000) {
            led_toggle(LD2_PIN);    // Toggle every 1000 msec
            _count = 0;
        }
        clear_UIF(TIM2);        // Clear TIM_UI flag
    }
}
```

# Part 2: PWM

## A. Register List

List of TIMx registers for PWM

**Figure 114. Output stage of capture/compare channel (channel 1)**




| Type | Register Name | Description |
|---|---|---|
| TIMx | TIMx _CCMRy | TIMx capture/compare mode register for yth channel |
| | TIMx _CCRy | TIMx capture/compare register for yth channel |
| | TIMx _CCER | TIMx capture/compare output enable register |

## B. Register Setting

### 1. System Clock setting

- Same as above

### 2. Timer Counter setting

- Same as above

### 3. **Timer Interrupt setting**

- If necessary

### 4. GPIO (AF) Out

- mode=AF(TIMx) for Pin_y in GPIOx

### 5. PWM Out setting

- Select PWM Output mode (TIMx->CCMR : OCyM)
- Select CompareCapture value (TIMx->CCRy : CCR)
- Select Output Polarity (TIMx->CCER : CCyP)
- Enable CompareCaptureOutput (TIMx->CCER : CCyE)

## C. Exercise

### Exercise: PWM Library

- Download the header library files and save under `include\`.
    - `ecPWM2_student.h, ecPWM2_student.c`: Click here to download
- Rename the files as `ecPWM2.h, ecPWM2.c`

> Write your name and modified date in the comment box

- Fill in the empty spaces in the downloaded library: `ecPWM2.c`
- Submit this tutorial report that shows the **Exercise Code**.

**Exercise Code**

```
// Internal Helper function: for PWM Initialization
void PWM_init_config(PinName_t pinName, uint32_t usec);
// Duty ratio 0~1.0
void PWM_duty(PinName_t pinName, float duty);
```

Exercise

```
////////////////////////////////////////////////
/*              EXERCISE                      */
////////////////////////////////////////////////

// Internal Helper function: for PWM Initialization
void PWM_init_config(PinName_t pinName, uint32_t usec){
// 0. Match TIMx from given Port_y and Pin_x
    GPIO_TypeDef *port;
    unsigned int pin;
    ecPinmap(pinName, &port, &pin);
    TIM_TypeDef *TIMx;
    int chN;
    PWM_pinmap(pinName, &TIMx, &chN);

// 1. Initialize GPIO port and pin as AF
    GPIO_init(pinName, AF);

// 2. Configure GPIO AFR by Pin x num.
    //  AFR[0] holds pin 0~7, AFR[1] holds pin 8~15 : 4 bits per pin
        uint32_t afrIdx   = pin >> 3;               // same as (pin/8)
    uint32_t afrShift = 4 * (pin % 8);
    uint32_t afNum    = 0;

    //  AF1 for TIM1,TIM2     AF2 for TIM3,TIM4
    if((TIMx == TIM1)||(TIMx == TIM2))      afNum = 1;  // AF1
    else if(TIMx== TIM3 || TIMx == TIM4)    afNum = 2;  // AF2
    // You can add more  for TIM5 to TIM11, if necessary [YOUR CODE GOES HERE!!]


    // Set AFR register for the pin
```

```
    port->AFR[afrIdx] &= ~(0xFUL << afrShift);  // 4 bits per pin
    // port->AFR[afrIdx] ____________________  ;
    // YOUR CODE GOES HERE!!]


// 3. Initialize Timer : select CLK_CNT according to PWM period
    //   TIM2/TIM5          (32-bit ARR) : 1 < usec < 2^32 usec     ->
TIM_init_us()
    //   TIM1/TIM3/TIM4     (16-bit ARR) : 1 < usec <=65,536 usec   ->
TIM_init_us()
    //                                     65 < msec <65,536 msec  -> TIM_init()
    if((TIMx == TIM2) || (TIMx == TIM5) || (usec <= 65536)){
        TIM_init_us(TIMx, usec);
    }
    else {
        uint32_t msec = usec / 1000;
        if(msec > 6553) msec = 6553;            // 16-bit ARR has limit of 6.5sec
        // [YOUR CODE GOES HERE!!]              // 65 < msec <65,536 msec   ->
TIM_init()

// 4. Configure Timer Output mode as PWM
    // Default: PWM Mode 1, active high, duty ratio = 50%
    PWM_mode_config(TIMx, chN, EC_PWM_MODE1);

// 5. Enable Timer Counter
    // For TIM1 ONLY
    if(TIMx == TIM1) TIMx->BDTR |= TIM_BDTR_MOE;                    // Main output
enable (MOE): 0 = Disable, 1 = Enable
    // All Timer: First, Clear the counter and Clear UI Flag
    TIMx->EGR |= TIM_EGR_UG;
    TIMx->SR   = ~TIM_SR_UIF;
    // Enable timers
    TIMx->CR1  |= TIM_CR1_CEN;

}


// Duty Ratio:  from 0.0 to 1.0
void PWM_duty(PinName_t pinName, float duty){
// 0. Match TIMx from  Port and Pin
    GPIO_TypeDef *port;
    unsigned int pin;
    ecPinmap(pinName, &port, &pin);
    TIM_TypeDef *TIMx;
    int chN;
    PWM_pinmap(pinName, &TIMx, &chN);

// 1. Check the duty ratio range from 0.0 to 1.0
    // [YOUR CODE GOES HERE!!]      // if duty < 0.0f  -> duty = 0.0f;
    // [YOUR CODE GOES HERE!!]      // if duty > 1.0f  -> duty = 1.0f;

// 2. Duty ratio -> CCR counts.  Assume PWM mode 1 :
    // uint32_t CCvalue = ____________________; // (ARR+1) * duty
    // [YOUR CODE GOES HERE!!]
```

```
    if(chN == 1)      { TIMx->CCR1 = CCvalue; } //set channel
    else if(chN == 2) { TIMx->CCR2 = CCvalue; }
    // REPEAT for CHn= 3, 4
    // [YOUR CODE GOES HERE!!]
    // [YOUR CODE GOES HERE!!]

}
```

ecPWM2.h

```
////////////////////////////////////////////////
/*              EXERCISE                      */
////////////////////////////////////////////////

// Internal Helper function: for PWM Initialization
void PWM_init_config(PinName_t pinName, uint32_t usec);
// Duty ratio 0~1.0
void PWM_duty(PinName_t pinName, float duty);


////////////////////////////////////////////////
/*      GIVEN:  PWM Initialization             */
////////////////////////////////////////////////

//  Low Frequency PWM initialization in msec. (below 1kHz)
//  Default:        50% duty ratio. PWM_mode1, Period in msec.
//  TIM2/TIM5       (32-bit ARR) : msec 1 ~ 4,294,967
//  TIM1/TIM3/TIM4  (16-bit ARR) : msec 1 ~ 6,553
void PWM_init(PinName_t pinName, uint32_t msec);    // 1 to 6,553 msec (16-bit)

//  High-Frequency PWM initialization in usec (1kHz to 100 kHz)
//  Default:        50% duty ratio.  PWM_mode1, Period in usec.
//  TIM2/TIM5       (32-bit ARR) :  usec 1 ~ 4,294,967
//  TIM1/TIM3/TIM4  (16-bit ARR) :  usec 1 ~ 6,553
void PWM_init_us(PinName_t pinName, uint32_t usec); // 10 usec and up


////////////////////////////////////////////////
/*      GIVEN: DUTY RATIO SETUP               */
////////////////////////////////////////////////

// Set Pulse width in msec
void PWM_pulsewidth(PinName_t pinName, uint32_t pulse_width_ms);

// Set Pulse width in usec
void PWM_pulsewidth_us(PinName_t pinName, uint32_t pulse_width_us);
```

filled in exercise ecPWM2.c file

```
/*----------------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : [ YOUR NAME GOES HERE !!!!!]
Created          : 05-03-2021
Modified         : 11-02-2026
Language/ver     : C++ in VS Code

Description      : PWM source code

/----------------------------------------------------------------*/

#include "ecPWM2.h"


////////////////////////////////////////////////
/*              EXERCISE                      */
////////////////////////////////////////////////


// Internal Helper function: for PWM Initialization
void PWM_init_config(PinName_t pinName, uint32_t usec){
// 0. Match TIMx from given Port_y and Pin_x
    GPIO_TypeDef *port;
    unsigned int pin;
    ecPinmap(pinName, &port, &pin);
    TIM_TypeDef *TIMx;
    int chN;
    PWM_pinmap(pinName, &TIMx, &chN);

// 1. Initialize GPIO port and pin as AF
    GPIO_init(pinName, AF);

// 2. Configure GPIO AFR by Pin x num.
    //  AFR[0] holds pin 0~7, AFR[1] holds pin 8~15 : 4 bits per pin
        uint32_t afrIdx   = pin >> 3;               // same as (pin/8)
    uint32_t afrShift = 4 * (pin % 8);
    uint32_t afNum    = 0;

    //  AF1 for TIM1,TIM2     AF2 for TIM3,TIM4
    if((TIMx == TIM1)||(TIMx == TIM2))      afNum = 1;  // AF1
    else if(TIMx== TIM3 || TIMx == TIM4)    afNum = 2;  // AF2
    // You can add more  for TIM5 to TIM11, if necessary [YOUR CODE GOES HERE!!]


    // Set AFR register for the pin
    port->AFR[afrIdx] &= ~(0xFUL << afrShift);  // 4 bits per pin
    // port->AFR[afrIdx] _____________________  ;
    port->AFR[afrIdx] |= (afNum << afrShift);


// 3. Initialize Timer : select CLK_CNT according to PWM period
    //   TIM2/TIM5          (32-bit ARR) : 1 < usec < 2^32 usec     ->
```

```
TIM_init_us()
    //   TIM1/TIM3/TIM4     (16-bit ARR) : 1 < usec <=65,536 usec    ->
TIM_init_us()
    //                                      65 < msec <65,536 msec  -> TIM_init()
    if((TIMx == TIM2) || (TIMx == TIM5) || (usec <= 65536)){
        TIM_init_us(TIMx, usec);
    }
    else {
        uint32_t msec = usec / 1000;
        if(msec > 6553) msec = 6553;            // 16-bit ARR has limit of 6.5sec
        TIM_init(TIMx, msec);
    }              // 65 < msec <65,536 msec   -> TIM_init()

// 4. Configure Timer Output mode as PWM
    // Default: PWM Mode 1, active high, duty ratio = 50%
    PWM_mode_config(TIMx, chN, EC_PWM_MODE1);

// 5. Enable Timer Counter
    // For TIM1 ONLY
    if(TIMx == TIM1) TIMx->BDTR |= TIM_BDTR_MOE;                    // Main output
enable (MOE): 0 = Disable, 1 = Enable
    // All Timer: First, Clear the counter and Clear UI Flag
    TIMx->EGR |= TIM_EGR_UG;
    TIMx->SR   = ~TIM_SR_UIF;
    // Enable timers
    TIMx->CR1  |= TIM_CR1_CEN;

}

// Duty Ratio:  from 0.0 to 1.0
void PWM_duty(PinName_t pinName, float duty){
// 0. Match TIMx from  Port and Pin
    GPIO_TypeDef *port;
    unsigned int pin;
    ecPinmap(pinName, &port, &pin);
    TIM_TypeDef *TIMx;
    int chN;
    PWM_pinmap(pinName, &TIMx, &chN);

// 1. Check the duty ratio range from 0.0 to 1.0
    if (duty < 0.0f) duty = 0.0f;       // if duty < 0.0f  -> duty = 0.0f;
    if (duty > 1.0f) duty = 1.0f;       // if duty > 1.0f  -> duty = 1.0f;

// 2. Duty ratio -> CCR counts.  Assume PWM mode 1 :
    // uint32_t CCvalue = ____________________; // (ARR+1) * duty
    uint32_t CCvalue = (uint32_t)((TIMx->ARR + 1) * duty);

    if(chN == 1)      { TIMx->CCR1 = CCvalue; } //set channel
    else if(chN == 2) { TIMx->CCR2 = CCvalue; }
    // REPEAT for CHn= 3, 4
    else if(chN == 3) { TIMx->CCR3 = CCvalue; } // set channel 3
    else if(chN == 4) { TIMx->CCR4 = CCvalue; } // set channel 4
}
```

```
////////////////////////////////////////////////
/*      GIVEN: PWM Initialization               */
////////////////////////////////////////////////

//  Low Frequency PWM initialization in msec. (below 1kHz)
//  Default:        50% duty ratio. PWM_mode1, Period in msec.
//  TIM2/TIM5       (32-bit ARR) : msec 1 ~ 4,294,967
//  TIM1/TIM3/TIM4  (16-bit ARR) : msec 1 ~ 6,553
void PWM_init(PinName_t pinName, uint32_t msec){
    if(msec == 0) msec = 1;     // minimum 1 msec
    PWM_init_config(pinName, msec * 1000);
}

//  High-Frequency PWM initialization in usec (1kHz to 100 kHz)
//  Default:        50% duty ratio.  PWM_mode1, Period in usec.
//  TIM2/TIM5       (32-bit ARR) :  usec 1 ~ 4,294,967
//  TIM1/TIM3/TIM4  (16-bit ARR) :  usec 1 ~ 6,553
void PWM_init_us(PinName_t pinName, uint32_t usec){
    if(usec < 10) usec = 10;  // Minimum 10us (100kHz)
    PWM_init_config(pinName, usec);
}


////////////////////////////////////////////////
/*      GIVEN: DUTY RATIO SETUP                 */
////////////////////////////////////////////////



// Set Pulse width in msec
void PWM_pulsewidth(PinName_t pinName, uint32_t pulse_width_ms){
// 0. Match TIMx from  Port and Pin
    GPIO_TypeDef *port;
    unsigned int pin;
    ecPinmap(pinName, &port, &pin);
    TIM_TypeDef *TIMx;
    int chN;
    PWM_pinmap(pinName, &TIMx, &chN);

    float duty=0.0f;
    uint32_t f_cnt =0;


// 1. Counter clock : CK_CNT = EC_SYSCLK / (PSC+1)
    f_cnt = (uint32_t)EC_SYSCLK / (TIMx->PSC + 1);      // [Hz]

// 2. Convert Pulse_width [msec] -> duty ratio (0~1.0)
    uint32_t counts = (uint32_t)(((uint64_t)f_cnt * pulse_width_ms) / 1000UL);
    duty = (float)counts / (float)(TIMx->ARR + 1);

// 3. Set duty ratio
```

```
    PWM_duty(pinName, duty);
}


// Set Pulse width in usec
void PWM_pulsewidth_us(PinName_t pinName, uint32_t pulse_width_us){
// 0. Match TIMx from  Port and Pin
    GPIO_TypeDef *port;
    unsigned int pin;
    ecPinmap(pinName, &port, &pin);
    TIM_TypeDef *TIMx;
    int chN;
    PWM_pinmap(pinName, &TIMx, &chN);

    float duty=0.0f;

// 1. Counter clock : CK_CNT = EC_SYSCLK / (PSC+1)
    uint32_t f_cnt = (uint32_t)EC_SYSCLK / (TIMx->PSC + 1);     // [Hz]

// 2. Convert Pulse_width [usec] -> duty ratio (0~1.0)
    // Exercise for Students
    uint32_t counts = (uint32_t)(((uint64_t)f_cnt * pulse_width_us) / 1000000UL);
    duty = (float)counts / (float)(TIMx->ARR + 1);

// 3. Set duty ratio
    PWM_duty(pinName, duty);
}


////////////////////////////////////////////////////////////////
/* Internal Helper Functions - GIVEN. DO NOT MODIFY           */
////////////////////////////////////////////////////////////////

// Internal Helper function for PWM output configuration
void PWM_mode_config(TIM_TypeDef *TIMx, int chN, uint32_t PWMmode){

    int mode=0x6; // default value for PWM mode 1
    if (PWMmode == EC_PWM_MODE2)    mode=0x7;

    uint32_t ccVal = TIMx->ARR/2;  // default value  CC=ARR/2
    if(chN == 1){
        TIMx->CCMR1 &= ~TIM_CCMR1_OC1M;                     // Clear ouput compare
mode bits for channel 1
        TIMx->CCMR1 |= ((mode << TIM_CCMR1_OC1M_Pos) & TIM_CCMR1_OC1M);    // OCxM
= 110 (PWM1) / 111 (PWM2) on ch1
        TIMx->CCMR1 |= TIM_CCMR1_OC1PE;                     // Output 1 preload
enable (make CCR1 value changable)
        TIMx->CCR1  = ccVal;                                // Output Compare
Register for channel 1 (default duty ratio = 50%)
        TIMx->CCER &= ~TIM_CCER_CC1P;                       // select output
polarity: active high
        TIMx->CCER  |= TIM_CCER_CC1E;                       // Enable output for
ch1
    }
```

```
    else if(chN == 2){
        TIMx->CCMR1 &= ~TIM_CCMR1_OC2M;                    // Clear ouput compare
mode bits for channel 2
        TIMx->CCMR1 |= ((mode << TIM_CCMR1_OC2M_Pos) & TIM_CCMR1_OC2M);   // OCxM
= 110 (PWM1) / 111 (PWM2) on ch2
        TIMx->CCMR1 |= TIM_CCMR1_OC2PE;                    // Output 1 preload
enable (make CCR1 value changable)
        TIMx->CCR2  = ccVal;                               // Output Compare
Register for channel 2 (default duty ratio = 50%)
        TIMx->CCER &= ~TIM_CCER_CC2P;                      // select output
polarity: active high
        TIMx->CCER |= TIM_CCER_CC2E;                       // Enable output for
ch2
    }
    else if(chN == 3){
        TIMx->CCMR2 &= ~TIM_CCMR2_OC3M;                    // Clear ouput compare
mode bits for channel 3
        TIMx->CCMR2 |= ((mode << TIM_CCMR2_OC3M_Pos) & TIM_CCMR2_OC3M);   // OCxM
= 110 (PWM1) / 111 (PWM2) on ch3
        TIMx->CCMR2 |= TIM_CCMR2_OC3PE;                    // Output 1 preload
enable (make CCR1 value changable)
        TIMx->CCR3  = ccVal;                               // Output Compare
Register for channel 3 (default duty ratio = 50%)
        TIMx->CCER &= ~TIM_CCER_CC3P;                      // select output
polarity: active high
        TIMx->CCER  |= TIM_CCER_CC3E;                      // Enable output for
ch3
    }
    else if(chN == 4){
        TIMx->CCMR2 &= ~TIM_CCMR2_OC4M;                    // Clear ouput compare
mode bits for channel 4
        TIMx->CCMR2 |= ((mode << TIM_CCMR2_OC4M_Pos) & TIM_CCMR2_OC4M);   // OCxM
= 110 (PWM1) / 111 (PWM2) on ch4
        TIMx->CCMR2 |= TIM_CCMR2_OC4PE;                    // Output 1 preload
enable (make CCR1 value changable)
        TIMx->CCR4  = ccVal;                               // Output Compare
Register for channel 4 (default duty ratio = 50%)
        TIMx->CCER &= ~TIM_CCER_CC4P;                      // select output
polarity: active high
        TIMx->CCER  |= TIM_CCER_CC4E;                      // Enable output for
ch4
    }
}


// Internal Helper function for PWM Pin Matching
void PWM_pinmap(PinName_t pinName, TIM_TypeDef **TIMx, int *chN)
{
    GPIO_TypeDef *port;
    unsigned int pin;
    ecPinmap(pinName, &port, &pin);

    // Only  TIM1 to TIM4 are defined.
    // TIM5, 9 to 11 needs to be added, if necessary
```

```
    if(port == GPIOA) {
        switch(pin){
           case 0 : *TIMx = TIM2; *chN    = 1; break;
           case 1 : *TIMx = TIM2; *chN = 2; break;
           case 5 : *TIMx = TIM2; *chN = 1; break;
           case 6 : *TIMx = TIM3; *chN = 1; break;
           //case 7: TIMx = TIM1; *chN = 1N; break;
           case 8 : *TIMx = TIM1; *chN = 1; break;
           case 9 : *TIMx = TIM1; *chN = 2; break;
           case 10: *TIMx = TIM1; *chN = 3; break;
           case 15: *TIMx = TIM2; *chN = 1; break;
           default: break;
        }
     }
     else if(port == GPIOB) {
        switch(pin){
           //case 0: TIMx = TIM1; *chN = 2N; break;
           //case 1: TIMx = TIM1; *chN = 3N; break;
           case 3 : *TIMx = TIM2; *chN = 2; break;
           case 4 : *TIMx = TIM3; *chN = 1; break;
           case 5 : *TIMx = TIM3; *chN = 2; break;
           case 6 : *TIMx = TIM4; *chN = 1; break;
           case 7 : *TIMx = TIM4; *chN = 2; break;
           case 8 : *TIMx = TIM4; *chN = 3; break;
           case 9 : *TIMx = TIM4; *chN = 4; break;
           case 10: *TIMx = TIM2; *chN = 3; break;
           default: break;
        }
     }
     else if(port == GPIOC) {
        switch(pin){
           case 6 : *TIMx = TIM3; *chN = 1; break;
           case 7 : *TIMx = TIM3; *chN = 2; break;
           case 8 : *TIMx = TIM3; *chN = 3; break;
           case 9 : *TIMx = TIM3; *chN = 4; break;
           default: break;
        }
     }
       // TIM5, 9, 11 needs to be added, if used.
      // [YOUR CODE GOES HERE!!] if necessary
}
```

### Exercise: PWM Application

This is an example code for PWM Out

- Give PWM output of 0% to 100% a step of 20%
- Check the brightness of LD2 (PA5) for each different Duty ratio

### Configuration

- System CLK is PLL 84MHz

- Timer Counter:
  - TIM2: Up-counting, Timer2_CLK = 100 kHz, COUNT_CLK=1 kHz
- PWM Output:
  - PWM Out on PA_5 : TIM2_CH1
  - PWM Mode 1 / Period 1kHz / Duty ratio 50%.

> There are several pins for TIM2_CH1. We will use GPIO Pin5 (LD2) for tutorial
>
> MCU pins for TIMERS

**Procedure**

- Create a new project under the directory `EC\tutorial\`
  - Environment : `env:TU_TIM_PWM`
  - Source file: `TU_TIM_PWM_student.c`
- Modify the `platformio.ini` , to add new environment.
- Run the program and check your result.
- No need to submit the result in the prelab report
- But you need to show the demonstration in class.

```
/*----------------------------------------------------------------@ Embedded
Controller by Young-Keun Kim - Handong Global University
Author           : [ YOUR NAME GOES HERE !!!!!]
Created          : 10-13-2026
Modified         : 00-00-2026 [WRITE THE DATE!!!!]
Language/ver     : C in VS Code

Description      : Tutorial - PWM
                   PA_5 (TIM2_CH1, Nucleo LD2) : 100 Hz, duty sweep 0 -> 100%
/----------------------------------------------------------------*/

// #include "ecSTM32F411.h"
#include "ecGPIO2.h"
#include "ecSysTick2.h"
#include "ecRCC2.h"
#include "ecTIM2.h"
#include "ecPWM2.h"

// Button Pin & PWM Pin
#define PWM_LED PA_5

// Initialiization
void setup(void) {
    RCC_PLL_init();
    SysTick_init();

    // PWM of 10 msec:  TIM2_CH1 (PA_5)
    PWM_init(PWM_LED, 10);          // 10 msec period -> 100 Hz
    PWM_duty(PWM_LED, 0.2f);        // 20% of duty
    // PWM_pulsewidth(PWM_LED, 2); //  a Pulse of 2msec =(duty=1/10=20%)
```

```
    // PWM of 1 msec:  TIM2_CH1 (PA_5)
    // PWM_init_us(PWM_LED, 1000);      // 1 msec period -> 1 kHz
    // PWM_duty(PWM_LED, 0.2f);     // 20% of duty
}



int main(void) {
    // Initialization --------------------------------------------------
    setup();

    // Infinite Loop ---------------------------------------------------
    while(1){
        for (int i=0; i<5; i++) {
            PWM_duty(PWM_LED, (float)0.2*i);
            delay_ms(1000);
        }
    }
}
```